Executive Summary

PROBLEM STATEMENT

Approximately **250 million students** in rural India attend government-aided schools following Maharashtra State Board (SSC) or CBSE/NCERT curricula. These students face a compound disadvantage: instruction is frequently delivered in a language that is not their mother tongue, textbooks assume consistent electricity and internet access, and pedagogical explanations rely on urban cultural references that are contextually inaccessible to agrarian communities.

ASER 2024 data confirms that **fewer than 42%** of Class 8 rural students in Maharashtra can solve a basic division problem. Science comprehension rates are lower still, with abstract concepts such as buoyancy, gravity, and refraction remaining opaque to students who lack relatable illustrative anchors.

TARGET USERS

■ PRIMARY USERS
Class 9 & 10 rural students following SSC Maharashtra or NCERT curricula in intermittent-connectivity zones.

■ SECONDARY USERS
Local teachers and community members who contribute village-specific cultural analogies to enrich the AI knowledge base.

■ TERTIARY USERS
School administrators and state education officials who can monitor active student engagement via the tracker panel.

KEY CHALLENGES ADDRESSED

■ Connectivity
Rural Maharashtra averages 2–4G coverage with frequent dropout. Standard SaaS AI tools are entirely unusable offline.

■ Language
Marathi-medium students lack AI tools that respond fluently in Marathi. English interfaces create significant cognitive overhead.

■ Cultural Context
Standard LLM outputs use Western references. Rural students need analogies from agricultural tools, village wells, and clay stoves.

■ Cost
Device heterogeneity is high. The system must run on mid-range Android browsers without requiring a dedicated app install.

PROPOSED SOLUTION

Grama-Gyan AI is a **Progressive Web Application (PWA)** that delivers a bilingual (Marathi/English) AI science tutor aligned with SSC Maharashtra and NCERT Class 9–10 syllabi. The system operates in **offline-first mode**, caches prior responses locally via IndexedDB, and injects culturally grounded village analogies into every LLM response via a purpose-built *Cultural Analogy Middleware*. Voice input and output via Bhashini (Government of India) removes literacy barriers for students with weaker reading fluency.

INNOVATION HIGHLIGHTS

■ Cultural Analogy Middleware
A keyword-matched analogy layer injects pre-seeded rural metaphors (village wells, bullock carts, clay stoves) into every LLM prompt before generation. Teachers can crowdsource new analogies live via the Gram-Gosht portal.

■ Offline-First RAG
Textbook chunks are bundled with the frontend. When MongoDB Atlas is unreachable, the system falls back to a local keyword-scored retrieval engine with zero degradation in curriculum accuracy.

■■ Vernacular Voice I/O
Bhashini ASR and TTS pipelines convert Marathi speech to text and synthesise spoken responses — critical for students with limited typing proficiency.

■ Curriculum-Grounded Responses
Every AI response cites exact textbook chapter and page numbers from SSC MH / NCERT textbooks, preventing hallucination and anchoring learning to the school syllabus.

PAGE 2 OF 5

System Architecture

HIGH-LEVEL LAYERED ARCHITECTURE

Layer	Technologies & Components
FRONTEND	React 19 PWA (Vite + TypeScript) | Tailwind CSS + Motion/Framer animations | **Service Worker** for asset caching | Browser Speech API (fallback ASR) | `IndexedDB` offline cache store
BACKEND	Node.js 20 + Express.js 4 REST API | Routes: `/api/query` `/api/asr` `/api/tts` | Middleware: rate limiter, input validator, CORS | Semantic MDS query-cache layer
AI / NLP	Gemini 2.5 Flash (primary LLM via @google/genai) | Groq LLM (low-latency fallback) | Bhashini ULCA ASR + TTS pipelines | Cultural Analogy Middleware | Keyword-scored RAG retrieval engine
DATABASE	MongoDB Atlas (cloud) — `TextbookChunk` `QueryCache` `CommunityAnalogy` collections | Mongoose ODM | Static textbook bundle (fallback) in `src/data/textbooks.js`
OFFLINE	IndexedDB — persistent conversation history | `localStorage` — student profile persistence | Bundled textbook chunks (static JSON) — zero-network RAG fallback | Browser TTS — voice fallback when Bhashini unreachable

REQUEST DATA FLOW — TEXT QUERY

Student Query Text/voice → **React PWA** IndexedDB check → **Express API** /api/query → **RAG Retrieval** MongoDB/static → **Gemini/Groq** Analogy prompt → **Response** Render+cache

REQUEST DATA FLOW — VOICE QUERY

Mic Input base64 audio → **POST /api/asr** Forward to Bhashini → **Bhashini ASR** Marathi speech→text → **Query Pipeline** RAG+Analogy+LLM → **POST /api/tts** Bhashini TTS→audio → **Audio Playback** Browser element

LAYER DESCRIPTIONS

REACT PWA (FRONTEND LAYER)

Single-page application built with React 19, Vite and TypeScript. Implements a PWA manifest and service worker enabling add-to-homescreen install without Play Store. Motion library provides micro-animations. All state (profile, conversation) persists to localStorage + IndexedDB on every update, ensuring data is not lost on page refresh.

NODE / EXPRESS (BACKEND LAYER)

RESTful API server with three primary route groups. The `query.service.js` orchestration pipeline coordinates RAG retrieval, analogy matching, prompt assembly, and LLM invocation as a sequential six-step pipeline. MDS hash-based semantic cache in MongoDB eliminates redundant LLM calls for identical queries.

GEMINI + GROQ (AI LAYER)

Gemini 2.5 Flash is the primary generative model; Groq serves as the low-latency fallback. All prompts are assembled by `promptBuilder.service.js`, which injects student profile, textbook context chunks, and a matched cultural analogy directive before generation. Prompt is strictly language-gated (Marathi or English).

MONGODB ATLAS (DATABASE LAYER)

Three Mongoose collections: `TextbookChunk` (indexed curriculum segments), `QueryCache` (MDS-keyed response cache), and `CommunityAnalogy` (teacher-submitted analogies). System degrades gracefully when Atlas is unreachable: static JS bundle handles retrieval with identical scoring logic.

Bhashini Integration (GoI NLTM)

India's Bhashini API provides government-hosted ASR and TTS for Marathi (ISO 639: mr). When Bhashini credentials are absent, the system falls back to the Web Speech API (browser-native), maintaining full voice I/O functionality at zero additional cost. The fallback path is fully implemented and tested in the MVP.

PAGE 3 OF 5

MVP Implementation

■ Implemented Features (Current MVP)

- Bilingual conversational AI tutor (Marathi / English)
- SSC MH & NCERT Class 9 & 10 chapter alignment (4 chapters + 1 NCERT)
- Voice input via Bhashini ASR (Web Speech API fallback)
- Voice output via Bhashini TTS (browser synth fallback)
- Cultural Analogy Middleware (keyword-match injection)
- Teacher analogy crowdsourcing portal (Gram-Gosht Yogdan)
- Offline conversation persistence (localStorage + IndexedDB)
- Textbook citation in every AI response (chapter + page)
- Student profile editor (name, village, board, class, language)
- Bookshelf / chapter navigation panel
- Suggested question chips (pre-seeded contextual prompts)
- Online/Offline connectivity indicator badge
- Semantic query cache (MDS hash, MongoDB+Atlas)
- Docker Compose deployment configuration
- Active Student Tracker panel (admin view)

■ Future Roadmap (Not Yet Built)

- Full vector-embedding semantic search (Atlas Vector / Pinecone)
- End-to-end Bhashini audio playback (full pipeline)
- Multi-subject support (Mathematics, History, Geography)
- Class 11 & 12 curriculum extension
- Real-time teacher dashboard with per-student analytics
- SMS / USSD fallback for feature phones (2G zones)
- Peer-to-peer local mesh sync (WebRTC)
- Push notifications for analogy approval workflow
- Persistent MongoDB vector index (full RAG v2)
- Multi-state expansion (Kannada, Telugu boards)
- National rollout via DIKSHA platform integration

APPLICATION SCREENSHOTS (MVP)

■■ AI Tutor Chat — Marathi Mode
Bilingual AI response with analogy injected; student: Priya from Sholgaon. Marathi interface with suggested question chips.

■■ AI Tutor Chat — English Mode
Same session switched to English. Language toggle in top navigation bar. Response regenerated in selected medium.

■■ Cultural Analogy Contribution Mode
Teacher-facing Gram-Gosht portal: submit scientific concept + regional analogy in Marathi. Contributed live to analogy pool.

■■ Student Profile Setup
Bilingual profile editor: name, village, board, class, grade, medium of study. Persisted to localStorage. Dropdown between selectors.

WORKFLOW A — VOICE QUERY (MARATHI)

1. Student taps **Mic button**. Browser `MediaRecorder` captures audio and encodes it to `base64`.

2. `POST /api/asr` forwards audio to **Bhashini ASR pipeline** (language: `mr`). If Bhashini credentials are absent, the **Web Speech API** handles transcription in-browser at zero cost.

3. Transcribed text enters the standard **six-step query pipeline**: hash check → RAG retrieval → analogy assembly → LLM generation.

4. Generated Marathi response is forwarded to `POST /api/tts`. Bhashini returns base64 audio; browser plays it via an element with a progress indicator.

WORKFLOW B — CULTURAL ANALOGY MIDDLEWARE

1. Incoming query text is normalised (lowercase). Keywords extracted: tokens with length > 2 characters.

2. `findAnalogyForQuery()` scans the `CULTURAL_ANALOGIES` array (pre-seeded + community-submitted) for keyword overlaps against `concept` and `keywords[]` fields.

3. If matched, the language-appropriate analogy text is appended to the **system instruction** as a weave-in directive. If unmatched, the LLM is instructed to generate a contextually appropriate rural analogy.

4. The response inherently includes the village analogy (e.g., *Like drawing water from a well, buoyancy acts upward against gravity…*). Analogy metadata is returned in the API response payload for UI display.

PAGE 4 OF 5

Technology & Scalability

TECHNOLOGY STACK

Component	Technology	Purpose / Justification
Frontend Runtime	React 19 + TypeScript + Vite	Strongly typed SPA; Vite delivers sub-second HMR and optimised production bundles. React 19 concurrent features reduce render blocking on low-end devices.
Progressive Web App	Service Worker + PWA Manifest	Enables add-to-homescreen on Android without Play Store. Caches strategic assets offline. Eliminates app installation friction for rural users.
Animations / UX	Motion (Framer Motion V2)	Smooth message entrances and model transitions. Improves perceived responsiveness. Automatically respects `prefers-reduced-motion`.
Offline Persistence	IndexedDB + localStorage	IndexedDB persists full conversation history; localStorage persists student profile. Both survive page refresh and browser restart without network.
Backend API	Node.js 20 + Express.js 4	Lightweight REST server serving both API and bundled frontend static files from a single port. MongoDB-optimised; rapid cold start.
Primary LLM	Google Gemini 2.5 Flash	Generous free-tier quota; competitive cost (~$0.075/1M tokens); fast inference; confirmed strong multilingual Marathi support confirmed in testing.
Fallback LLM	Groq (Llama / Mistral)	Sub-100ms inference on Groq LPU hardware. Activates on Gemini quota exhaustion or latency spikes. API compatible, enabling transparent failover.
Speech — ASR	Bhashini ULCA API (GoI)	Government-backed Indian-language ASR; Marathi model fine-tuned on Indian dialect data. Fallback: Web Speech API (browser-native, zero cost).
Speech — TTS	Bhashini TTS + Browser SpeechSynthesis	Bhashini generates natural Marathi TTS. Browser `SpeechSynthesis` activates when Bhashini credentials are absent, maintaining voice output at zero operational cost.
Database	MongoDB Atlas + Mongoose	Document model suits semi-structured textbook chunks and variable-length AI responses. Atlas free tier (512 MB) sufficient for MVP pilot. Graceful offline fallback to static JSON bundle.
Ingestion Pipeline	Node.js (`scripts/ingest_*.js`)	Parse and chunk SSC MH / NCERT textbook content; designed to generate a new textbook entry per new textbook.
Containerisation	Docker + Docker Compose	Single `docker-compose.yml` starts backend + static server. Ensures reproducible deployment across developer environments and CI pipelines.
Deployment Target	Vercel (FE) + Railway (BE)	Vercel CDN edge frontends globally. Railway provides persistent Node.js process for backend. Both have generous free tiers appropriate for pilot scale.

SCALABILITY ANALYSIS

■ Offline-First Hierarchy
Three-tier fallback: MongoDB Atlas → static textbook bundle → local IndexedDB. No single failure point breaks the learning experience. Students in network-dead zones receive full curriculum responses.

■■ Query Cache Layer
Semantic caching via MDS hash of `(query + class + language)` eliminates redundant LLM API calls. Estimated to reduce LLM cost by 60–80% for common curriculum queries.

■ Dual LLM Failover
Gemini Flash → Groq automatic failover prevents service outages. Median response latency remains under 3 s for both providers under common network conditions.

■ Multi-State Expansion
Analogy system, ingestion pipeline, and Bhashini integration are language-parameterised. Karnataka support requires: (i) Kannada analogy seeds, (ii) KSEEB textbook ingestion, (iii) Bhashini language code switch.

PERFORMANCE & COST SUMMARY

Metric	Current MVP	At 10,000 Daily Users
Avg. response latency (online)	1.8 – 3.2 s	~2–4 s with CDN caching
Avg. response latency (offline)	< 200 ms (cached)	< 200 ms (no network dependency)
LLM cost (Gemini Flash)	~$0.075 / 1M tokens	~$3–6/day post-cache (est.)
MongoDB storage	< 50 MB (MVP)	~400 MB–500 MB
Frontend bundle size	~420 KB gzipped	No change (static asset)
Bhashini ASR / TTS	Free (GoI-funded)	Free up to quota; SLA via MeitY

Future Deployment: Integration with India's DIKSHA platform (Ministry of Education) would enable zero-cost distribution to 260 million enrolled students. DIKSHA already operates PWA-based content delivery; Grama-Gyan AI's architecture is deliberately designed for this compatibility.

PAGE 5 OF 5

Impact, Validation & Roadmap

250M+	42%	<35%	22+
Rural school students in India (potential addressable base)	Class 8 rural students who can't solve a basic division (ASER 2024)	Rural Class 9–10 science comprehension rate (est. from ASER data)	Scheduled Indian languages supported by Bhashini U/A pipeline

POLICY & RESEARCH ALIGNMENT

■ ASER 2024 Relevance
ASER 2024 documents critical science literacy gaps in rural Maharashtra. Grama-Gyan AI directly addresses the comprehension deficit by grounding abstract concepts in familiar agrarian vocabulary — the primary intervention recommended by ASER researchers for this demographic.

■ NEP 2020 Alignment
NEP 2020 mandates mother-tongue medium instruction through Class 5 and encourages multilingual pedagogical approaches beyond. Grama-Gyan is the first AI tutor purpose-built for this policy requirement, supporting Marathi-medium SSC students at secondary level with a bilingual interface.

■ Mother-Tongue Learning
Cognitive science research shows 20–30% higher retention when concepts are first encountered in the learner's mother tongue. Grama-Gyan's Marathi-first interface, combined with culturally resonant analogies, applies this evidence base in a production-grade software system.

EXPECTED IMPACT

■ Student-Level Outcomes
- Reduced conceptual confusion for physics and biology topics via Village context analogies
- 24/7 AI tutoring access, eliminating dependency on teacher availability in single-teacher schools
- Voice-first interaction removes the literacy barrier for students who struggle with typing
- Offline mode ensures learning continuity during 60–80% of hours that lack reliable internet

■■ Systemic Outcomes
- Teacher crowdsourcing captures and digitises indigenous pedagogical knowledge currently held only in oral tradition
- Curriculum-grounded citations train students in textbook navigation — a transferable academic skill
- Analogy database self-improves as teachers contribute, creating a compounding knowledge effect
- Offline-first architecture is cost-sustainable (with caching) without per user subscription fees

DEVELOPMENT ROADMAP

Phase 1 — Months 0–3 (Current)
MVP Validation
- Deploy to 2–3 pilot schools in rural Maharashtra
- Collect teacher analogy contributions (target: 100)
- Validate Bhashini ASR on rural Marathi dialects
- End-to-end UI testing with Class 9 & 10 content
- Performance profiling on low-end Android devices
- Publish open-source analogy dataset (CC-BY license)

Phase 2 — Months 4–9
Platform Hardenscale
- Upgrade to vector embedding RAG (Atlas Vector Search)
- Add Mathematics and History subject modules
- Real-time teacher dashboard with per student analytics
- Class 11 & 12 curriculum extension
- Multi-school admin panel
- MeitY DIKSHA content delivery API integration
- Kannada / Telugu board support (Phase 2B)

Phase 3 — Months 10–18
State Education Dept. partnership
- DIKSHA platform integration (260M students)
- SMS / USSD mode for feature phones (20-only devices)
- Peer-to-peer local mesh sync via local Wi-Fi / Bluetooth
- Formal impact assessment with randomised control trial
- Open analogy API for third-party curriculum applications

Grama-Gyan AI is a functional, deployed MVP demonstrating a technically sound approach to closing the rural science education gap in India. The system combines offline-first architecture, government-backed language infrastructure (Bhashini), a novel Cultural Analogy Middleware, and curriculum-grounded AI generation into a single installable PWA accessible on any Android browser. Technology choices are conservative, cost-effective, and policy-aligned. The engineering innovation — the Analogy Middleware and its community crowdsourcing loop — is novel, reproducible, and extendable to any Indian language or state curriculum. Grama-Gyan is not a prototype concept; it is a working system ready for field validation.

QUICK REFERENCE

■ Repository
github.com/anoopcodehack/Grama-Gyan-AI
License: MIT | Open Source

■ Required API Keys
```
GEMINI_API_KEY
BHASHINI_API_KEY
GROQ_API_KEY
MONGODB_URI
```

■ Quick Start
```
npm install
npm run dev
```
Serves on Port 3000